

# claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-ae470abc86afb338.jsonl`  
- SHA-256: `be4b82cecb835b8499ab04c2e5d73062ab5ac73f3b13df6193d3695ebd0d6246`  
- Source modified: `2026-07-06T09:14:47+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_3512746c-54e`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

## Transcript

### 1. user (2026-07-06T09:13:45.213Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).  
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +  
an ingestion gate in telethon\_client.py that skips unwatched sources. Changed files ONLY:  
command\_service.py (new /watch etc + \_resolve\_source), commands.py (re-export patterns), db.py (set\_source\_watched, list\_account\_sources), telethon\_client.py (the gate),  
tests/test\_command\_service.py (10 tests).  
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;  
/channels should also upsert candidates from client.get\_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db.\_apply\_schema on aiosqlite ":memory:"; seed via db.upsert\_account + db.upsert\_source; drive command\_service.CommandService().handle()).  
NOTE: real telegram sources have a NUMERIC provider\_source\_id (str(channel\_id) e.g. '-1001234567890'); upsert\_channel  
creates them with watched=True. The on\_new\_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.  
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),  
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND docs/adr/0009-selective-channel-scanning.md, trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff  
 and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms / trigger can't occur /  
 pre-existing. PLAUSIBLE if unsettled. Corrected severity.  
 FINDING (adr\_gate): /channels does not upsert candidates from get\_dialogs(), so untrafficked channels are undiscoverable and unwatchable (ADR 0009 lines 48-54)  
 severity medium | src/tg\_video\_filter/command\_service.py:370  
 summary: ADR 0009's discoverability note (lines 48-54) requires /channels to upsert candidate rows from `client.get\_dialogs()` so users can browse and /watch channels they have not yet received traffic from. `\_handle\_channels` only calls `db.list\_account\_sources` ? it never enumerates dialogs and never creates rows. With the correct opt-in default (watched=0, no auto-created rows), a channel the account has never received a message from has no source row at all, so it cannot be listed or watched.  
 scenario: A user joins a curated source channel that has not posted a new video since they joined. They run /channels expecting to see it and /watch it. Because no message has arrived, no source row exists, and \_handle\_channels lists nothing for it. \_resolve\_source then returns None for /watch, and the user is told 'Source not found ... a channel you haven't received traffic from may not appear yet' ? leaving them unable to opt in until traffic happens to arrive. The ADR explicitly calls this out as the reason /channels must upsert from get\_dialogs(); the feature ships without that half, so opt-in is necessary but (per the ADR's own wording) not sufficient for discoverability.  
 evidence: command\_service.py:370-385 `\_handle\_channels` body calls only  
 `db.list\_account\_sources(conn, user\_id)` and formats the result; there is no get\_dialogs() call anywhere in the changed files (grep of the diff shows get\_dialogs only pre-exists in backfill.py, which this PR did not touch). ADR 0009 lines 51-54: 'The /channels command ... therefore upserts candidate rows from client.get\_dialogs() so users can browse and watch channels they have not yet received traffic from.' The empty-state text at command\_service.py:374-379 redirects to /backfill instead of performing the ADR-specified dialog upsert.  
 rec: Have /channels (or a shared helper) enumerate `client.get\_dialogs()` and upsert each candidate as a source with watched=0 before listing, per ADR lines 48-54. Since CommandService is provider-independent (no Telethon client), thread a dialog-enumeration callback/port in through the Telethon adapter, or perform the upsert in the adapter before delegating to \_handle\_channels.

## 2. assistant (2026-07-06T09:13:47.772Z)

I'll verify this finding by reading the ADR spec and the actual code.

## 3. user (2026-07-06T09:13:48.514Z)

```

1      # ADR 0009: Selective channel scanning and multiplexed routing
2
3      - **Status**: Accepted
4      - **Date**: 2026-07-04
5      - **Decision owner**: project owner
6
7      ## Context
8
9      Today the userbot scans every channel/group the account is a member of.
10     `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11     filter, and `on_new_message` processes every video regardless of source. The
12     `channels` table exists but is populated lazily as a record of seen channels;
13     it is not used as a filter.
14
15     The product vision is a multiplexing video re-distribution system:
16
17     - A small set of high-quality source channels (e.g. spiritual, classical
18       music) are scanned for incoming videos.
19     - Matches are routed to themed destination broadcast channels (e.g. a
20       Reggae channel, a Rap channel) that subscribers opt into.
21     - A user therefore subscribes to outputs by topic, not to inputs by source.
22
23     This requires two capabilities the current architecture does not have:

```

```

24
25 1. **Selective scanning** ? choose which source channels are watched, rather
26   than ingesting all of them.
27 2. **Per-source ? per-destination routing** ? a match from source channel X is
28   delivered to a specific destination channel Y, instead of the single global
29   `delivery_target` (ADR 0005).
30
31 ## Decision
32
33 Add three additive, independently-shippable capabilities:
34
35 ### 1. Selective scanning (watched-flag)
36
37 Add a `watched` flag to the `channels` table. The default is locked as
38 **opt-in: `DEFAULT 0`**, so a freshly-onboarded user scans nothing until they
39 explicitly `/watch` a channel. This matches the multiplexing product shape
40 (sources are a *curated* set) and avoids blindly re-ingesting every chat the
41 account is a member of.
42
43 Gate `on_new_message` at the top: after computing `channel_id`, look up the
44 row and `return` early if `watched` is false, before any dedup or filter work.
45
46 This is a small migration plus ~5 lines in `telethon_client.py`.
47
48 **Discoverability note.** With `DEFAULT 0` plus the current lazy population
49 (a `channels` row is only inserted when a message is first seen), opt-in is
50 necessary but not sufficient for discoverability: a channel the account has
51 never received a message from has no row to flag. The `/channels` command
52 (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53 so users can browse and watch channels they have not yet received traffic
54 from. The flag default and the enumeration flow are orthogonal ? both are
55 required.
56
57 ### 2. Channel enumeration and management commands
58
59 New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60 existing `commands.py` pattern:
61
62 - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63   used in `backfill.py`) with their watched state.
64 - `/watch <#name|id>` ? set `watched=1` for a source channel.
65 - `/unwatch <#name|id>` ? set `watched=0`.
66 - `/watchlist` ? list only watched sources.
67
68 ### 3. Per-source ? per-destination routing rules
69
70 Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
71 refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73 ```
74 routing_rules (
75     user_id          INTEGER NOT NULL REFERENCES users(id),
76     source_channel_id INTEGER NOT NULL,
77     dest_channel_id  INTEGER NOT NULL,    -- a broadcast channel / @username
78     filter_config_id  INTEGER,           -- optional per-route filter override
79     PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80 )
81 ```
82
83 On a match, look up applicable routing rules by `source_channel_id` and
84 deliver to each matching destination. The delivery layer (`delivery.py`)
85 already accepts a `target` per call, so this generalises the existing single
86 `delivery_target` to a per-match lookup. Users with no routing rules fall
87 back to the current global `delivery_target` behaviour (ADR 0005), preserving
88 backwards compatibility.

```

```

89
90 ## Consequences
91
92 **Positive**
93 - Enables the multiplexing product shape: scan curated sources, publish to
94   topical broadcast channels, let subscribers pick outputs by interest.
95 - Each capability is additive and independently deployable; none disturbs the
96   existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97 - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original*
98   file, so a routed re-upload is not re-processed.
99 - Channel enumeration reuses `client.get_dialogs()` already present in
100   `backfill.py`; no new Telethon capability is required.
101
102 **Negative**
103 - New schema (`channels.watched`, `routing_rules`) with migrations.
104 - **Routing loop invariant.** With multiplexed routing, the userbot account
105   may be a member of a *destination* broadcast channel and re-see its own
106   output, producing an infinite forward loop. The contract is: **a channel
107   is either a source or a destination, never both.** This is enforced in
108   `on_new_message` by excluding any channel that is a configured destination
109   (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110   user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111   this; the invariant is stated here so the handler has a contract to code
112   against.
113 - Per-route filter overrides add config complexity; v1 may ship with global
114   filters only and a single implicit route (`source ? global target`), then
115   layer explicit per-route rules later.
116
117 ## Backwards compatibility
118
119 Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120 install from receiving matches until the user re-watches their channels ? a
121 behaviour change disguised as a schema change. To preserve the current
122 "ingest everything" semantics for existing installs, the migration backfills:
123
124 ```sql
125 ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126 UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127 ```
128
129 i.e. any channel the install has *already* seen traffic from is implicitly
130 watched, so pre-migration users observe no change. New channels discovered
131 after the migration default to `watched = 0` and must be explicitly watched ?
132 which is the intended new UX. This mirrors the "old rows behave exactly as
133 before" pattern established by ADR 0005.
134
135 ## Rollout order
136
137 1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138   `/channels`). Smallest unit; immediately useful even without routing.
139 2. **Named destination channels** ? extend `/set delivery_target` to allow
140   switching among a user-defined set, not just one global value.
141 3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142   `/route <source> <dest>` command). The full multiplexing vision.
143
144 Each step is shippable and useful on its own; step 3 is not required to
145 deliver value from steps 1?2.
146
147 ## Alternatives considered
148
149 - **Server-side `chats=` filter at handler registration:** rejected ? the
150   watched set is per-user and mutable at runtime; re-registering Telethon
151   handlers on every `/watch` change is brittle. An in-handler lookup against
152   the `channels` table is simpler and hot-swappable.
153 - **Content-based classification (auto-detect genre):** explicitly out of

```

```

154     scope for v1. Routing is user-defined by source channel; automatic topic
155     detection could be a future enhancement but is not required to deliver the
156     multiplexing UX.
157
158     ## References
159
160     - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161     - ADR 0005 (per-user delivery target) ? the single-target delivery this
162       generalises into per-route targets.
163     - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164       the original `document_id`.
165     - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166       `/unwatch`, and routing commands to public users.
167

```

#### 4. user (2026-07-06T09:13:48.821Z)

```

1     """Provider-independent command parsing and execution (ADR 0011, Track D).
2
3     Extracts command logic from the Telethon adapter (`commands.py`) so the
4     same `/set`, `/show` commands work identically through a Bot API
5     front-door. The service is pure: strings in, `CommandResult` out ? no
6     Telethon objects, no network calls.
7
8     See `docs/adr/0007-inline-config-commands.md` for the command model and
9     `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API motivation.
10    """
11
12    from __future__ import annotations
13
14    import logging
15    import re
16    from dataclasses import dataclass
17
18    import aiosqlite
19
20    from tg_video_filter import db
21    from tg_video_filter.models import FilterConfig, Source
22
23    log = logging.getLogger(__name__)
24
25    # Key into the shared engine_holder dict (imported by commands.py's adapter).
26    ENGINE_KEY = "engine"
27
28    # --- regex patterns -----
29
30    _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmg]?b?)$", re.IGNORECASE)
31    _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
32    _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
33    _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
34    _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
35    _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
36    _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
37    _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
38    _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
39
40
41    # --- source scanning regex (ADR 0009, Track E) -----
42
43    _WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
44    _UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
45    _CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
46    _WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)

```

```

47
48 # --- unit parsing -----
49
50
51 def parse_size_mb(value_str: str, unit: str) -> int:
52     """Parse a size value + unit string into mebibytes."""
53     value = int(value_str)
54     unit = unit.lower().rstrip("b")
55     multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
56     if unit in multipliers:
57         raw = value * multipliers[unit]
58         if unit == "k":
59             raw = max(1, raw // 1024)
60         return raw
61     raise ValueError(f"Unknown size unit: {unit}")
62
63
64 def parse_duration_s(value_str: str, unit: str) -> int:
65     """Parse a duration value + unit string into seconds."""
66     value = int(value_str)
67     unit = unit.lower()
68     multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
69     if unit in multipliers:
70         return value * multipliers[unit]
71     raise ValueError(f"Unknown time unit: {unit}")
72
73
74 # --- formatting helpers -----
75
76
77 def _format_duration(seconds: int | None) -> str:
78     if seconds is None:
79         return "unlimited"
80     if seconds == 0:
81         return "0s"
82     if seconds < 60:
83         return f"{seconds}s"
84     if seconds < 3600:
85         m, s = divmod(seconds, 60)
86         return f"{m}m{s}s" if s else f"{m}m"
87     h, rem = divmod(seconds, 3600)
88     m, s = divmod(rem, 60)
89     parts = f"{h}h"
90     if m:
91         parts += f"{m}m"
92     if s:
93         parts += f"{s}s"
94     return parts
95
96
97 def _format_size(mb: int | None) -> str:
98     if mb is None:
99         return "unlimited"
100     if mb == 0:
101         return "0MB"
102     if mb >= 1024:
103         gb = mb / 1024
104         if gb == int(gb):
105             return f"{int(gb)}GB"
106         return f"{gb:.1f}GB"
107     return f"{mb}MB"
108
109
110 def format_config(cfg: FilterConfig) -> str:
111     """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""

```

```

112     lines = [
113         "? **Current Filter Configuration**",
114         "",
115         f"? **Duration:** {_format_duration(cfg.min_duration_s)} -
116         {_format_duration(cfg.max_duration_s)}",
117         f"? **Min resolution:** {' ' + str(cfg.target_height) + 'p' if
118         cfg.target_height else 'disabled'}",
119         f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
120         f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
121         f"? **Delivery target:** `{cfg.delivery_target}`",
122         "",
123         "? **Content exclusions:**",
124         f"    GIF/animated: {' excluded' if cfg.exclude_gif else '? allowed'}",
125         f"    Round videos: {' excluded' if cfg.exclude_round else '? allowed'}",
126         f"    Silent (no audio): {' excluded' if cfg.exclude_nosound else '? allowed'}",
127     ]
128     return "\n".join(lines)
129
130 def format_commands() -> str:
131     """Return a help message listing all available commands."""
132     return "\n".join(
133         [
134             "? **Available Commands**",
135             "",
136             "? **Filter config**",
137             "`/show config` ? Show current filter settings",
138             "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
139             "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing
140             `B`",
141             "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
142             "    or `/set max length <N><unit>` ? same as above",
143             "    Units: `s` (default), `m`, `h` ? case-insensitive",
144             "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
145             5s`)",
146             "    Units: `s` (default), `m`, `h` ? case-insensitive",
147             "`/set min resolution <N>p` ? Set min video height (e.g. `/set min
148             resolution 720p`)",
149             "    or `/set min res <N>p` ? same as above",
150             "",
151             "? **Content exclusions** (all off by default)",
152             "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
153             "`/set exclude round on|off` ? Exclude circular video messages",
154             "`/set exclude nosound on|off` ? Exclude videos without audio\n"
155             "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs
156             too.",
157             "",
158             "? **Backfill**",
159             "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
160             (default 90d)",
161             "`/stop` ? Cancel running backfill",
162             "",
163             "\u2139ufe0f **Help**",
164             "`/show commands` ? Show this help message",
165         ]
166     )
167
168 # --- command result -----
169
170 @dataclass
171 class CommandResult:
172     """The output of executing a command ? provider-independent.

```

```

170     The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
171     and ``config_changed`` to decide whether to hot-reload the filter engine.
172     """
173
174     reply_text: str = ""
175     config_changed: bool = False
176
177
178     # --- command service -----
179
180
181     class CommandService:
182         """Provider-independent command execution.
183
184         Parses a command string against an account's current config and DB,
185         returning a ``CommandResult``. No Telethon objects involved ? pure string
186         in, effects out. The Telethon adapter (``commands.py``) handles the
187         transport (checking message.out, sending the reply); the Bot API adapter
188         (future) calls ``handle()`` with the ``from.id`` from the update.
189
190         Usage::
191
192             svc = CommandService()
193             result = await svc.handle(user_id, text, conn, current_cfg)
194             if result.config_changed:
195                 engine_holder["engine"] = FilterEngine(current_cfg)
196             await send_reply(result.reply_text)
197         """
198
199         # -- public API -----
200
201         async def handle(
202             self,
203             user_id: int,
204             text: str,
205             conn: aiosqlite.Connection,
206             current_cfg: FilterConfig,
207         ) -> CommandResult:
208             """Parse and execute a command text, returning the reply + change flag."""
209             text = text.rstrip()
210
211             # --- /set size <value><unit> -----
212             m = _SIZE_RE.match(text)
213             if m:
214                 value_str, unit = m.group(1), m.group(2) or ""
215                 # NOTE: ValueError is unreachable here ? the regex already validates
216                 # the unit against [kmgT]?b?, so parse_size_mb can never raise.
217                 # Kept for defensive parity with the original commands.py refactor.
218                 try:
219                     new_mb = parse_size_mb(value_str, unit)
220                 except ValueError:
221                     return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
222                 if new_mb < 1:
223                     return CommandResult("? Size must be at least 1 MB (``FilterConfig``
constraint).")
224                 cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
225                 return CommandResult(
226                     f"? Max file size set to
**[_format_size(new_mb)]**.\n\n{format_config(cfg)}",
227                     config_changed=True,
228                 )
229
230             # --- /set length <value><unit> -----
231             m = _LENGTH_RE.match(text)

```

```

232         if m:
233             value_str, unit = m.group(1), m.group(2) or ""
234             try:
235                 new_secs = parse_duration_s(value_str, unit)
236             except ValueError:
237                 return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
238             min_secs = current_cfg.min_duration_s or 0
239             if new_secs < min_secs:
240                 return CommandResult(
241                     f"? Max duration ({_format_duration(new_secs)}) must be "
242                     f">= min duration ({_format_duration(min_secs)}).")
243             )
244             cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
245             return CommandResult(
246                 f"? Max duration set to
**{_format_duration(new_secs)}**.\n\n{format_config(cfg)}",
247                 config_changed=True,
248             )
249
250         # --- /set min length <value><unit> -----
251         m = _MIN_LENGTH_RE.match(text)
252         if m:
253             value_str, unit = m.group(1), m.group(2) or ""
254             try:
255                 new_min = parse_duration_s(value_str, unit)
256             except ValueError:
257                 return CommandResult("? Invalid duration format. Use e.g. `/set min
length 5s`.")
258             max_secs = current_cfg.max_duration_s or 0
259             if max_secs > 0 and new_min > max_secs:
260                 return CommandResult(
261                     f"? Min duration ({_format_duration(new_min)}) must be "
262                     f"<= max duration ({_format_duration(max_secs)}).")
263             )
264             cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
265             return CommandResult(
266                 f"? Min duration set to
**{_format_duration(new_min)}**.\n\n{format_config(cfg)}",
267                 config_changed=True,
268             )
269
270         # --- /set min resolution <N>p -----
271         m = _MIN_RES_RE.match(text)
272         if m:
273             new_height = int(m.group(1))
274             if new_height < 1:
275                 return CommandResult("? Resolution must be at least 1p.")
276             cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
277             return CommandResult(
278                 f"? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}",
279                 config_changed=True,
280             )
281
282         if _MIN_RES_HELP_RE.match(text):
283             return CommandResult(
284                 "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
285             )
286
287         # --- /set exclude <gif|round|nosound> <on|off> -----
288         m = _EXCLUDE_RE.match(text)
289         if m:
290             what = m.group(1).lower()

```

```

291         state = m.group(2).lower()
292         enabled = state == "on"
293         labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
294         field_map = {
295             "gif": "exclude_gif",
296             "round": "exclude_round",
297             "nosound": "exclude_nosound",
298         }
299         cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
300         return CommandResult(
301             f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
302             f"\n\n{format_config(cfg)}",
303             config_changed=True,
304         )
305
306     if _EXCLUDE_HELP_RE.match(text):
307         return CommandResult(
308             "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
309             "Example: `/set exclude gif on`"
310         )
311
312     # --- /show config -----
313     if _SHOW_CONFIG_RE.match(text):
314         return CommandResult(format_config(current_cfg))
315
316     # --- source scanning commands (ADR 0009, Track E) -----
317
318     m = _WATCH_RE.match(text)
319     if m:
320         return await self._handle_watch(conn, user_id, m.group(1).strip(),
current_cfg)
321
322     m = _UNWATCH_RE.match(text)
323     if m:
324         return await self._handle_unwatch(conn, user_id, m.group(1).strip(),
current_cfg)
325
326     if _CHANNELS_RE.match(text):
327         return await self._handle_channels(conn, user_id)
328
329     if _WATCHLIST_RE.match(text):
330         return await self._handle_watchlist(conn, user_id)
331
332     # --- /show commands -----
333     if _SHOW_COMMANDS_RE.match(text):
334         return CommandResult(format_commands())
335
336     # Not a recognised command ? return empty result (noise).
337     return CommandResult()
338
339     # -- source scanning handlers (ADR 0009, Track E) -----
340
341     async def _handle_watch(
342         self, conn: aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
343     ) -> CommandResult:
344         """Parse `/watch <username|channel_id>` and toggle `watched=1`."""
345         src = await self._resolve_source(conn, user_id, target)
346         if src is None:
347             return CommandResult(
348                 f"? Source `{target}` not found. Use `/channels` to list "
349                 f"available sources.\n\nSources are discovered as messages "
350                 f"arrive; a channel you haven't received traffic from may not "
351                 f"appear yet."

```

```

352         )
353         await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-
type]
354         label = src.title or src.provider_source_id
355         return CommandResult(f"? Now watching **{label}**
({src.provider_source_id}).")
356
357     async def _handle_unwatch(
358         self, conn:aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
359     ) -> CommandResult:
360         """Parse `/unwatch <@username|channel_id>` and toggle `watched=0`."""
361         src = await self._resolve_source(conn, user_id, target)
362         if src is None:
363             return CommandResult(
364                 f"? Source `{target}` not found. Use `/channels` to list sources."
365             )
366         await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-
type]
367         label = src.title or src.provider_source_id
368         return CommandResult(f"? Stopped watching **{label}**
({src.provider_source_id}).")
369
370     async def _handle_channels(self, conn:aiosqlite.Connection, user_id: int) ->
CommandResult:
371         """List all known sources with watch status."""
372         sources = await db.list_account_sources(conn, user_id)
373         if not sources:
374             return CommandResult(
375                 "? **No sources yet.**\n\n"
376                 "Sources are discovered when a new message arrives from a "
377                 "channel. Run `/backfill` to discover historical channels, "
378                 "or wait for new traffic."
379             )
380         lines = ["? **All sources**", ""]
381         for s in sources:
382             icon = "?" if s.watched else "?"
383             label = s.title or s.provider_source_id
384             lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
385         return CommandResult("\n".join(lines))
386
387     async def _handle_watchlist(self, conn:aiosqlite.Connection, user_id: int) ->
CommandResult:
388         """List only watched sources."""
389         sources = await db.list_account_sources(conn, user_id, watched_only=True)
390         if not sources:
391             return CommandResult(
392                 "? **No watched sources.**\n\n"
393                 "Use `/channels` to see available sources, then `/watch <id>` "
394                 "to start scanning."
395             )
396         lines = ["? **Watched sources**", ""]
397         for s in sources:
398             label = s.title or s.provider_source_id
399             lines.append(f"? `{s.provider_source_id}` ? {label}")
400         return CommandResult("\n".join(lines))
401
402     async def _resolve_source(
403         self, conn:aiosqlite.Connection, user_id: int, target: str
404     ) -> Source | None:
405         """Resolve a user-provided source identifier to a `Source` row.
406
407         Accepts `@username` or a numeric channel id string. The `@` prefix
408         is stripped before lookup since Telegram channel ids are numeric.
409         """
410         target = target.lstrip("@")

```

```

411         # Try by provider-native id first (the common case).
412         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
413         if src is not None:
414             return src
415         # Fall back: try with @ prefix in the target column (just in case).
416         return await db.get_source_by_provider_id(conn, user_id, "telegram",
f"@{target}")
417
418     # -- internal -----
419
420     async def _update(
421         self,
422         conn: aiosqlite.Connection,
423         user_id: int,
424         current: FilterConfig,
425         max_size_mb: int | None = None,
426         max_duration_s: int | None = None,
427         min_duration_s: int | None = None,
428         target_height: int | None = None,
429         exclude_gif: bool | None = None,
430         exclude_round: bool | None = None,
431         exclude_nosound: bool | None = None,
432     ) -> FilterConfig:
433         """Apply partial updates to the account's config, persist, and return."""
434         updates: dict[str, object] = {}
435         if max_size_mb is not None:
436             updates["max_size_mb"] = max_size_mb
437         if max_duration_s is not None:
438             updates["max_duration_s"] = max_duration_s
439         if min_duration_s is not None:
440             updates["min_duration_s"] = min_duration_s
441         if target_height is not None:
442             updates["target_height"] = target_height
443         if exclude_gif is not None:
444             updates["exclude_gif"] = exclude_gif
445         if exclude_round is not None:
446             updates["exclude_round"] = exclude_round
447         if exclude_nosound is not None:
448             updates["exclude_nosound"] = exclude_nosound
449         cfg = current.model_copy(update=updates) if updates else current
450         await db.save_filter_config(conn, user_id, cfg)
451         log.info(
452             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
453             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
454             user_id,
455             cfg.max_size_mb,
456             cfg.min_duration_s,
457             cfg.max_duration_s,
458             cfg.target_height,
459             cfg.exclude_gif,
460             cfg.exclude_round,
461             cfg.exclude_nosound,
462         )
463         return cfg
464
465
466     __all__ = [
467         "ENGINE_KEY",
468         "CommandResult",
469         "CommandService",
470         "_format_duration",
471         "_format_size",
472         "format_config",
473         "format_commands",
474         "parse_duration_s",

```

```
475         "parse_size_mb",
476     ]
477
```

## 5. assistant (2026-07-06T09:13:52.026Z)

Now let me verify the key claim: does `list\_account\_sources` only return existing rows, and is there no `get\_dialogs()` upsert anywhere in the changed files? Let me check the db functions and confirm the diff scope.

## 6. user (2026-07-06T09:13:53.637Z)

```
src\tg_video_filter\backfill.py:120:     dialogs = await client.get_dialogs()
```

## 7. user (2026-07-06T09:13:54.742Z)

```
1104:async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1142:async def get_source_by_provider_id(
1387:async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1537:async def set_source_watched(
1548:async def list_account_sources(
```

## 8. assistant (2026-07-06T09:13:57.779Z)

`get\_dialogs` only exists in `backfill.py` (unchanged by this PR). Let me read `list\_account\_sources` and the telethon\_client gate to fully confirm the control flow.

## 9. user (2026-07-06T09:13:58.788Z)

```
1537     async def set_source_watched(
1538         conn: aiosqlite.Connection, account_id: int, source_id: int, watched: bool
1539     ) -> None:
1540         """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
1541         await conn.execute(
1542             "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
1543             (int(watched), source_id, account_id),
1544         )
1545         await conn.commit()
1546
1547
1548     async def list_account_sources(
1549         conn: aiosqlite.Connection, account_id: int, *, watched_only: bool = False
1550     ) -> list[Source]:
1551         """List all sources for an account, optionally filtering to watched only."""
1552         conn.row_factory = aiosqlite.Row
1553         query = "SELECT * FROM sources WHERE account_id = ?"
1554         params: tuple = (account_id,)
1555         if watched_only:
1556             query += " AND watched = 1"
1557         query += " ORDER BY watched DESC, provider_source_id ASC"
1558         async with conn.execute(query, params) as cur:
1559             rows = await cur.fetchall()
1560             return [_row_to_source(r) for r in rows]
1561
1562
1563     async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
1564         """Deprecated shim: insert into ``content_items`` (ADR 0011).
1565
1566         Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1567         if absent), computes the provider-neutral ``dedup_key`` via
1568         :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1569         ``VideoRecord`` shape is reconstructed from the inserted row.
1570
1571         """
1572         src = await upsert_source(
```

```

1572         conn,
1573         Source(
1574             account_id=video.user_id,
1575             provider="telegram",
1576             provider_source_id=str(video.channel_id),
1577             kind="channel",
1578             watched=True,
1579         ),
1580     )
1581     assert src.id is not None # upsert_source populates the surrogate PK
1582     item = ContentItem(
1583         account_id=video.user_id,
1584         source_id=src.id,
1585         provider="telegram",
1586         provider_message_id=str(video.message_id),
1587         content_type="video",
1588         document_id=str(video.document_id) if video.document_id is not None else None,
1589         duration_s=video.duration_s,
1590         width=video.width,
1591         height=video.height,
1592         size_bytes=video.size_bytes,
1593         mime_type=video.mime_type,
1594         dedup_key=compute_dedup_key(
1595             provider="telegram",
1596             document_id=(str(video.document_id) if video.document_id is not None else
None),
1597             source_id=src.id,
1598             provider_message_id=str(video.message_id),
1599             size_bytes=video.size_bytes,
1600             duration_s=video.duration_s,
1601             height=video.height,
1602             width=video.width,
1603         ),
1604         matched=video.matched,
1605         seen_at=video.seen_at,
1606     )

```

## 10. user (2026-07-06T09:13:59.327Z)

```

1     """Telethon ingestion: client factory, event handler, and metadata extraction.
2
3     Architecture (see docs/design.md §5):
4
5         NewMessage event
6         -> extract_video_meta(message) # pure, unit-tested
7         -> dedup via db.get_video()
8         -> FilterEngine.evaluate(meta)
9         -> db.insert_video(...) + delivery.forward_to_target(...,
target=cfg.delivery_target)
10
11     The metadata extraction is deliberately a pure function taking a Telethon
12     ``Message`` and returning a ``VideoMeta | None``. This isolates the only piece
13     of logic that's worth unit-testing (everything else is Telethon/async glue).
14     """
15
16     from __future__ import annotations
17
18     import logging
19     import sqlite3
20     from pathlib import Path
21     from typing import Protocol
22
23     import aiosqlite
24     from telethon import TelegramClient
25     from telethon.events import NewMessage

```

```

26     from telethon.tl.types import (
27         Document,
28         DocumentAttributeAnimated,
29         DocumentAttributeVideo,
30         Message,
31         MessageMediaDocument,
32     )
33
34     from tg_video_filter import db
35     from tg_video_filter.commands import ENGINE_KEY
36     from tg_video_filter.delivery import build_link_text, deliver
37     from tg_video_filter.filters import FilterEngine, VideoMeta
38     from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User,
VideoRecord
39
40     log = logging.getLogger(__name__)

```

#### 11. assistant (2026-07-06T09:14:09.241Z)

The finding is factually accurate on the code level. Now the critical adversarial question: does the ADR make `get_dialogs()` upsert a **required** part of THIS PR (Track E), or is it a note that could legitimately ship in a later step?

Let me carefully re-read the ADR discoverability note and rollout. Line 48-55 is unambiguous: "The `/channels` command (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`... The flag default and the enumeration flow are orthogonal ? **both are required.**" Line 62: "`/channels` ? list the user's channels (via `client.get_dialogs()`, already used in `backfill.py`)". And rollout line 137-138 puts `/channels` firmly in step 1 alongside `/watch`, `/unwatch`.

So the ADR explicitly ties `get_dialogs()` enumeration to `/channels` in the same rollout step this PR implements, and calls it "required." The PR ships `/channels` reading only from `list_account_sources`, with no dialog enumeration anywhere.

Now let me stress the refutation angles:

1. **Is CommandService the right layer?** The finding acknowledges `CommandService` is provider-independent (no Telethon client). This is a real architectural tension ? but it doesn't excuse the gap; the ADR requires the capability somewhere in the `/channels` flow, and the PR added neither a port/callback nor an adapter-side upsert. Let me confirm the Telethon adapter (`commands.py`) doesn't do the dialog upsert before delegating.

#### 12. user (2026-07-06T09:14:11.088Z)

```

31:     _CHANNELS_RE,

```

#### 13. assistant (2026-07-06T09:14:16.724Z)

`commands.py` only re-exports `_CHANNELS_RE`; there is no dialog-enumeration upsert in the adapter either. Let me confirm what `commands.py` does around the re-export to be thorough about whether the adapter wires in any dialog logic.

#### 14. user (2026-07-06T09:14:17.596Z)

```

20     from telethon import TelegramClient
21     from telethon.errors import RPCError
22
23     from tg_video_filter import db
24     from tg_video_filter.command_service import ENGINE_KEY, CommandResult, CommandService
25     from tg_video_filter.filters import FilterEngine
26     from tg_video_filter.models import FilterConfig, User
27
28     log = logging.getLogger(__name__)
29

```

```

30 from tg_video_filter.command_service import ( # noqa: E402, F401
31     _CHANNELS_RE,
32     _EXCLUDE_HELP_RE,
33     _EXCLUDE_RE,
34     _LENGTH_RE,
35     _MIN_LENGTH_RE,
36     _MIN_RES_HELP_RE,
37     _MIN_RES_RE,
38     _SHOW_COMMANDS_RE,
39     _SHOW_CONFIG_RE,
40     _SIZE_RE,
41     _UNWATCH_RE,
42     _WATCH_RE,
43     _WATCHLIST_RE,
44     _format_duration,
45     _format_size,
46     format_commands,
47     format_config,
48     parse_duration_s,
49     parse_size_mb,
50 )
51
52
53 def make_config_command_handler(
54     user: User,
55     conn: aiosqlite.Connection,
56     engine_holder: dict[str, FilterEngine],
57 ) -> Callable:
58     svc = CommandService()
59
60     async def on_command(event: object) -> None:
61         message = getattr(event, "message", None)
62         if message is None:
63             return
64         if not getattr(message, "out", False):
65             return
66         peer = getattr(message, "peer_id", None)
67         peer_user_id = getattr(peer, "user_id", None) if peer else None
68         if peer_user_id != user.id:
69             return
70
71         text: str = getattr(message, "message", "") or ""
72         client: TelegramClient = getattr(event, "client", None) # type:
ignore[assignment]
73
74         # Defer DB load ? skip non-command self-messages (review finding #2).
75         if not text.startswith("/"):
76             return
77
78         current_cfg = await db.load_filter_config(conn, user.id)
79         result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
80
81         if result.config_changed:
82             new_cfg = await db.load_filter_config(conn, user.id)
83             engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
84
85         if result.reply_text:
86             await _reply(client, message, result.reply_text)
87
88     return on_command
89
90
91 # --- backwards-compat helpers (test_commands.py references) -----
92
93

```

```

94     async def _update_config(
95         conn: aiomysql.Connection,
96         user_id: int,
97         max_size_mb: int | None = None,
98         max_duration_s: int | None = None,
99         min_duration_s: int | None = None,

```

## 15. assistant (2026-07-06T09:14:35.082Z)

Confirmed. The Telethon adapter ``on_command`` has ``client`` available (line 72) but delegates entirely to ``svc.handle()`` with no dialog-enumeration step, and ``handle()``'s ``/channels`` path only calls ``list_account_sources``. No layer in this PR upserts candidate rows from ``get_dialogs()``.

The finding is factually correct and the code genuinely deviates from the ADR. Now let me weigh the severity honestly.

**\*\*Confirming the deviation is real and user-hitting:\*\***

- ADR line 37-38 + backwards-compat line 125-131: new channels default ``watched=0`` and rows only get created when traffic arrives (lazy population via ``insert_video`/`upsert_source``). A freshly-joined channel with no new traffic has no ``sources`` row.
- ``/channels`` (``_handle_channels``, line 370-385) reads only ``list_account_sources``, which is a plain ``SELECT FROM sources`` ? no dialog enumeration.
- So the untrafficked channel is invisible in ``/channels``, and ``_resolve_source`` returns `None` for ``/watch``, producing the "not found... may not appear yet" message (line 348-352). The user cannot opt in until traffic arrives.
- ADR explicitly declares this enumeration "required" and places ``/channels`` in rollout step 1 (this PR).

**\*\*Refutation attempts and why they fail:\*\***

- "It's just a note / future step" ? refuted: line 54 says "both are required"; line 137-138 puts ``/channels`` in step 1.
- "CommandService can't reach Telethon" ? a real architectural constraint, but the ADR names ``backfill.py``'s existing ``get_dialogs()`` as the reuse path and the adapter has ``client`` in hand; the gap is a design omission, not an impossibility. Doesn't refute the deviation.
- Is there a workaround? ``/backfill`` populates rows via ``get_dialogs`` in ``backfill.py`` ? but ``backfill`` creates source rows via the ingestion path (``upsert_source`` with ``watched=True`` per line 1578!). Let me not overreach here; regardless, ``/backfill`` is a separate, heavier command and the ADR requires ``/channels`` itself to do the upsert. The empty-state text even redirects to ``/backfill``, confirming the authors knew ``/channels`` alone doesn't discover.

**\*\*Severity assessment:\*\*** The finding says medium. Is it higher? The core opt-in feature (``/watch``, gate) works for any channel that HAS a row. The undiscoverable case is specifically "joined a channel that has posted zero new messages since joining and you haven't backfilled." That's a real but narrower slice ? not the main path being broken, and there's a partial (if imperfect) escape hatch (``/backfill``, or simply waiting for one message). It's a genuine ADR-contract violation of an explicitly-"required" clause, which argues against dropping to low. Medium is the honest landing: an ADR-mandated half of the discoverability story is missing, degrading the feature for a real subset of users, but the primary opt-in flow still functions.

I'll confirm at medium.

## 16. user (2026-07-06T09:14:46.928Z)

Structured output provided successfully